Solution: Palindrome Permutation

Problem Statement

Given a string `s`, return `true` if any permutation of the given string `s` can form a palindromic string. Otherwise, return `false`.

A palindrome is a word, phrase, number, or other sequences of characters that reads the same forward and backward (ignoring spaces, punctuation, and capitalization).

Examples

- **Example 1**:
    - **Input**: `"tactcoa"`
    - **Expected Output**: `true`
    - **Justification**: The string can be rearranged to form "tacocat", which is a palindrome.
- **Example 2**:
    - **Input**: `"abcde"`
    - **Expected Output**: `false`
    - **Justification**: There is no possible rearrangement of these characters that would result in a palindrome.
- **Example 3**:
    - **Input**: `"aabbccdd"`
    - **Expected Output**: `true`
    - **Justification**: The string can be rearranged as "abcddcba", which is palindromes.

Solution

To solve this problem, the key is to understand the characteristics of a palindrome. In a palindrome, every character must appear an even number of times, except for at most one character, which can appear an odd number of times (in the case of strings with an odd length). Thus, our approach will involve counting the frequency of each character in the string and ensuring that no more than one character has an odd count.

This approach is effective because it directly addresses the fundamental property of palindromes. By focusing on character counts, we can efficiently determine the potential for a palindrome permutation without needing to generate permutations, which would be computationally expensive. The algorithm's simplicity and directness make it an ideal solution for this problem.

Step-by-step Algorithm

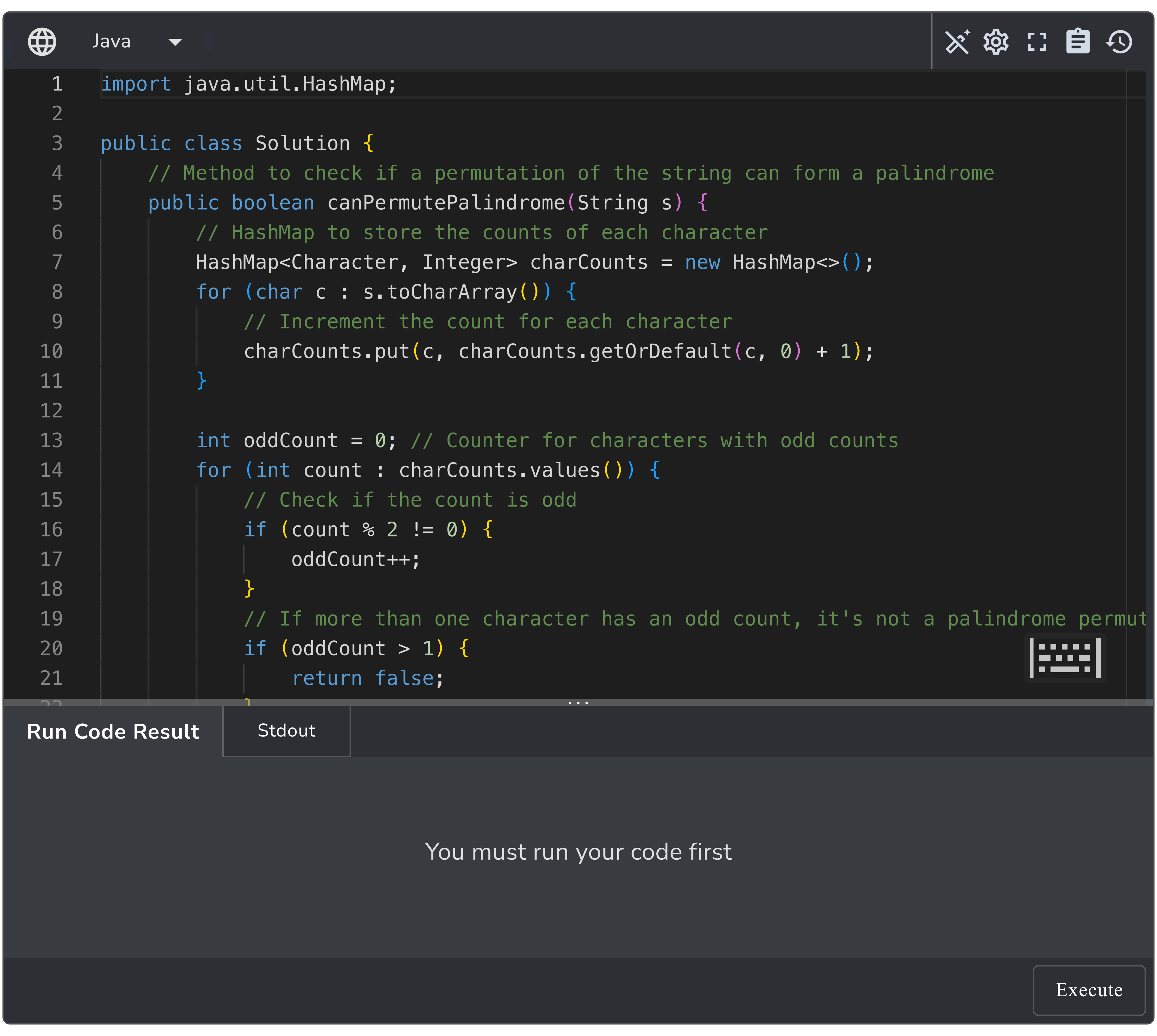
- Initialize a hash map or an array to count the occurrences of each character in the string.
- Iterate over each character in the string:
    - Increment the count for each character in the hash map/array.
- Initialize a variable to count the number of characters that appear an odd number of times.
- Iterate over the hash map/array:
    - If a character's count is odd, increment the odd count variable.
- Check the odd count variable:
    - **If it is greater than 1, return** `false`.
    - Otherwise, **return** `true`.

Algorithm Walkthrough

Let's walk through the algorithm using the input `"tactcoa"`:

1. **Initialize Counter:** Create a hash map for character counts.
2. **Count Characters:**
    - For `"tactcoa"`, count each character.
3. **Hash Map Result:**
    - Counts are `{'t': 2, 'a': 2, 'c': 2, 'o': 1}`.
4. **Count Odd Occurrences:**
    - Only 'o' has an odd count (1).
5. **Check Condition:**
    - Only one odd count, so return `true`.

Code



```java
import java.util.HashMap;

public class Solution {
    // Method to check if a permutation of the string can form a palindrome
    public boolean canPermutePalindrome(String s) {
        // HashMap to store the counts of each character
        HashMap<Character, Integer> charCounts = new HashMap<>();
        for (char c : s.toCharArray()) {
            // Increment the count for each character
            charCounts.put(c, charCounts.getOrDefault(c, 0) + 1);
        }

        int oddCount = 0; // Counter for characters with odd counts
        for (int count : charCounts.values()) {
            // Check if the count is odd
            if (count % 2 != 0) {
                oddCount++;
            }
            // If more than one character has an odd count, it's not a palindrome permut
            if (oddCount > 1) {
                return false;
```

You must run your code first

Complexity Analysis

- **Time Complexity**: The time complexity of this algorithm is `O(n)`, where n is the length of the string, because we need to iterate through each character to count occurrences.
- **Space Complexity**: The space complexity is `O(1)` as the hash map/array will have a fixed size determined by the character set (in case of ASCII, it's constant).

← Previous
Palindrome Permutation (Easy)
Next →
Rank Transform Of An Array
☐ Mark as Completed